PRACTICAL – 12

Write a program for travelling salesman problem.

Aim:
To write a Python program to solve the Travelling Salesman Problem (TSP) and find the minimum cost tour that visits every city exactly once and returns to the starting city.

Objective:
To find the minimum-cost tour for a set of cities by checking all possible tours.

Algorithm:
1. Start.
2. Initialize the cost matrix containing the distances between cities.
3. Set the minimum cost to infinity.
4. Generate all possible routes starting from the first city.
5. Calculate the total cost of each route.
6. Add the cost of returning to the starting city.
7. If the total cost is less than the minimum cost, update the minimum cost and best route.
8. Repeat until all possible routes are checked.
9. Display the minimum cost and the best tour.
10. Stop.

Code:
```
from itertools import permutations

n = 4
INF = 99999

cost = [
   [0, 10, 15, 20],
   [10, 0, 35, 25],
   [15, 35, 0, 30],
   [20, 25, 30, 0]
]

cities = list(range(1, n))
min_cost = INF
best_path = []
```

```
for path in permutations(cities):
    total = cost[0][path[0]]

    for i in range(len(path) - 1):
        total += cost[path[i]][path[i + 1]]

    total += cost[path[-1]][0]

    if total < min_cost:
        min_cost = total
        best_path = [0] + list(path) + [0]

print("Minimum Cost:", min_cost)
print("Best Tour:", " -> ".join(str(city + 1) for city in best_path))
print("EN.No: 92460118673")
```

Output:

```
PS C:\Users\vatsalya\Downloads>  & 'C:\Users\vatsalya\AppData\Local\Programs\
'  'c:\Users\vatsalya\Downloads\main (2).py'
Minimum Cost: 80
Best Tour: 1 -> 2 -> 4 -> 3 -> 1
EN.No: 92460118673
PS C:\Users\vatsalya\Downloads>
```

Time Complexity:

The program checks all possible tours. There are \((n-1)!\) possible tours, and calculating the cost of each tour takes O(n) time.
Overall time complexity: O(n!)

Space Comp	*Time Complexity*	*Operation*
O(n^2)	O(n^2)	Initialize cost matrix
O(n)	O((n-1)!)	Generate all possible tours
O(n)	O(n!)	Calculate tour cost and find minimum